

HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY



Lab 12: Design Principles

ITSS Software Development: IT4549E

Students	Student ID
Bùi Trung Hiếu	20235932
Nguyễn Hữu Tuấn Tú	20236005
Nguyễn Tuấn Khanh	20235952
Nguyễn Trần Bình	20235899
Trần Trung Nghĩa	20235983

Semester 20252

1. Single Responsibility Principle (SRP)

Modules	PIC	Description	Improvement Direction
PlaceOrderService	Khanh NT	Bears 7 responsibilities simultaneously: (1) cart item normalization and input validation, (2) delivery info validation, (3) payment info validation, (4) stock availability checking, (5) invoice, VAT, and shipping fee calculation, (6) database persistence — creating Order, Invoice, OrderProduct, Transaction records and decrementing stock, (7) email notification. Any change to the email template, VAT rate, validation rules, or DB schema forces modification of this single class.	Extract OrderValidationService (validation), InvoiceCalculationService (VAT + shipping fee), OrderPersistenceService (DB writes). Keep PlaceOrderBeService as a thin orchestrator. Move email notification to an event-driven pattern.
PlaceOrderPayment Adapter	Khanh NT	Bundles 4 responsibilities: (1) DB query to provide payment context (getPaymentContext), (2) order status transition PENDING_PAYMENT → PENDING_PROCESSING, (3) stock decrement for every ordered product, (4) building InvoicePreviewDto and sending confirmation email. Each responsibility has independent reasons to change.	Extract OrderStatusService (status transition + stock decrement) and OrderNotificationService (invoice building + email). Keep the adapter as a thin bridge between payment callbacks and the order domain.
VietqrService	Bình NT	Partial violation. The class is still provider-focused, but it handles multiple responsibilities at once: (1) outbound token caching, (2) VietQR credential/config lookup, (3) QR request mapping, (4) QR generation, (5) callback response formatting, (6) sandbox	Extract VietqrTokenService for access-token retrieval/cache, VietqrConfigService for typed environment config, and move payment callback status mapping to the payment layer. Keep VietqrService focused on provider-specific QR operations.

		callback testing. A change to token handling, QR payload format, callback response format, or sandbox behavior can force modification of this single class.	
VietqrInboundService	Binh NT	Low risk. The class has one broad responsibility: VietQR inbound authentication. Inside that boundary it still performs several steps: (1) parse Basic Auth, (2) validate merchant credentials, (3) create signed temporary token, (4) parse callback token, (5) verify signature and expiry. If the token format or signing algorithm changes, this class must change.	Extract InboundTokenSigner / InboundTokenVerifier. Keep credential validation separate from token creation/verification.
VietqrController	Binh NT	Low risk. The controller remains inside the VietQR boundary, but it exposes production QR generation, production callback synchronization, and sandbox callback testing in one class. Production hardening and sandbox behavior have different reasons to change.	Split sandbox endpoint into VietqrSandboxController, registered only in non-production environments. Keep VietqrController for production QR and callback endpoints.
PaymentService VietQR flow	Binh NT	Medium risk. The service coordinates transaction persistence, provider-specific payment request creation, VietQR callback confirmation, order status update through PlaceOrderPaymentPort, and refund-required handling. These responsibilities change for different business reasons.	Keep PaymentService as a transaction orchestrator. Move provider-specific payment creation and callback mapping to PaymentGatewayPort strategies.
VietQrPaymentFlowService	Binh NT	Medium risk. The service handles (1) creating VietQR	Extract PaymentSnapshotStore, PaymentPollingService, and

		payment, (2) polling transaction status, (3) triggering sandbox callback, (4) storing/restoring payment snapshots in browser storage, (5) deriving gateway order id. Polling, storage, and sandbox testing can change independently.	VietQrSandboxCallbackService. Keep this service focused on coordinating the VietQR payment flow.
PaymentPageFacade	Binh NT, Tu NHT	Medium risk. The facade coordinates checkout draft validation, invoice preview loading, VietQR payment state, router navigation, page state mutation, and QR image generation. Changes to QR rendering, invoice preview, or navigation can all force modification here.	Extract QrImageService, PaymentPageStateStore, and InvoicePreviewFacade. Keep PaymentPageFacade focused on composing page-level interactions.
DeliveryComponent	Khanh NT	Medium risk. The component owns reactive form setup, field error display, draft persistence, invoice preview loading, VietQR payment creation, navigation, and price formatting. UI validation, payment flow, and preview calculation are independent reasons to change.	Extract DeliveryFormFacade and PlaceOrderCheckoutFacade. Keep the component responsible mainly for template binding and user events.
CheckoutDraftService	Khanh NT	Low risk. The service is centered on checkout draft lifecycle, but it persists drafts, parses stored JSON, validates selected products, and maps draft items to place-order DTOs.	Extract CheckoutDraftStorage, CheckoutDraftValidator, and optionally CheckoutDraftMapper if draft logic grows.
ProductService	HieuBT	Handles too many responsibilities: creating products, querying the database, uploading images, deleting, and checking the daily delete limit for the user	Split into smaller services like: ProductCommandService, ProductQueryService,... and move the policy/user-related logic to DailyQuotaService.

		(deleteProduct). The deleteProduct function checks the policy, updates the database, decides whether to delete/deactivate, and returns the result.	
CreateProductDto	HieuBT	God DTO that combines all fields for all product types (Book, CD, DVD, Newspaper). This file will need to be modified whenever any product type changes or when new products are added (Clothing, E-Book).	Create a BaseCreateProductDto containing common fields, and create separate subclasses like CreateBookDto that inherit from the base.
ProductFormLogic (Client)	HieuBT	Manages form state, contains form initialization logic, handles image uploads, aggregates payloads, and calls APIs all within a single class. The `saveProduct` and `updateSpecificInfoForm` functions handle different processing for each product type, making the class complex.	Split the API call data mapping part into a PayloadMapper (Strategy), and use the form config base to decouple the form logic.
ProductDetailMapper (api – application/mappers)	NghiaTT	Medium risk. A single mapper class both assembles ProductDetailDto/specificInfo for all four product types (Book, Newspaper, CD, DVD) and owns low-level value parsing/normalization (parseDimensions, parseNamedDimensions, normalizeType, normalizeStatus, toNumber, toDateString, toStringList, mapAuthors). DTO shape and raw-data parsing rules change for different reasons, yet both pull edits into this one class.	Extract a ProductValueParser / FieldNormalizer (dimensions, dates, numbers, lists) and keep ProductDetailMapper focused only on persistence-record → DTO field assembly.
ProductDetailCom	NghiaTT	Partial violation. The	Extract an AddToCartControl

ponent (client)		component mixes two responsibilities: rendering common product-detail fields and owning the add-to-cart interaction (quantity signal, increase/decreaseQuantity, canAddToCart, addToCart) plus building status-notice text. Presentation layout and cart behavior change for independent reasons	component (quantity selector + addToCart) and move statusnotice text to a helper/pipe; keep ProductDetailComponent for display only.
ProductDetailPage Component (client)	NghiaTT	Low risk. Besides coordinating route-param reading and loading/error state, the page also hosts several navigation helpers (goToProducts, goToCatalog, goToCart) and inline HTTPError mapping, slightly widening its reasons to change.	Keep the page as a thin coordinator; move navigation into a small router helper and centralize HTTP-error → message mapping.
PaypalGatewayAdapter	Tu NHT	Handles exactly one responsibility: Adapting data from PaymentGatewayContext to the format required by PaypalService, and formatting the returned PaymentGatewayResult.	Currently performing its Adapter role well. Continue maintaining this as a thin class focused purely on data mapping.
PaypalService	Tu NHT	Bears multiple responsibilities simultaneously: (1) Managing and reading environment variables (process.env), (2) Instantiating and executing direct HTTP requests via the 3rd-party axios library, (3)	- Extract configuration reading (use NestJS ConfigService). - Extract direct HTTP calls (use NestJS HttpModule/HttpService and

		Handling PayPal API business logic (token processing, payload structure). Any change in how environment variables are configured or replacing the HTTP client library forces modification of this single class.	inject it). - Keep PayPalService focused solely on preparing payloads and processing PayPal responses.
--	--	---	---

2. Open/Closed Principle (OCP)

Modules	PIC	Description	Improvement Direction
PlaceOrderService.sendOrderConfirmationEmail()	Khanh NT	Notification is hardcoded to email only. According to AIMS additional requirements, future channels include SMS, Zalo, and push notifications. Every new channel requires modifying this method and adding a new concrete dependency - violating "closed for modification".	Introduce INotificationService { notify(recipient, content) }. Inject it instead of MailService. Add new channels by implementing the interface; PlaceOrderService never changes.
ShippingFeeService (shared, used by place-order)	Khanh NT	Only one fixed algorithm (weight-based). The AIMS additional requirements specify a volumetric weight strategy: $MAX(actual\ weight, L \times W \times H / 6000)$. Adding it requires modifying calculateShippingFee() — no abstraction exists for the strategy.	Introduce IShippingFeeStrategy { calculate(...): number }. Current logic becomes WeightBasedShippingStrategy. Add VolumetricWeightShippingStrategy without touching this class.
DeliveryInfoValidator.validate() (shared)	Khanh NT	Every new validation rule (e.g., province whitelist, enhanced email format) requires directly modifying this class. There is no plugin/extension mechanism	Define IValidationRule { validate(info): string null }. DeliveryInfoValidator accepts a list of rules and executes them; adding a rule means adding a class, not changing the

		for new rules.	validator.
PaymentService.requestPayment()	Binh NT	Clear violation. Payment provider selection is hardcoded with branches for VIETQR and PAYPAL. Adding a new provider such as COD, credit card, or another bank gateway requires modifying PaymentService.	Introduce PaymentGatewayPort with provider strategies. Register gateways in a provider registry/map and let PaymentService delegate without changing its core logic.
VietqrController.testCallback()	Binh NT	Partial violation. Sandbox callback behavior is hardcoded in the same controller as production endpoints. Disabling sandbox in production or changing test-only behavior requires modifying this controller.	Move sandbox callback endpoint to VietqrSandboxController and register it only for development/test environments.
VietQrPaymentFlowService.requestSandboxCallback()	Binh NT	Partial violation. Sandbox callback is hardcoded into the VietQR payment flow. Changing environment-specific behavior requires editing the class.	Add an environment flag or injectable VietQrCallbackStrategy. Production can use a no-op strategy; sandbox can use the test callback strategy.
DeliveryComponent.submit()	Khanh NT	Partial violation. The component directly starts VietQR payment. Adding another payment method requires changing this component.	Introduce checkout/payment flow abstraction, e.g. PlaceOrderCheckoutFacade or PaymentFlowStrategy selection, so the component delegates payment creation.
PaymentStatusApiService	Binh NT	Low risk. The API service mixes production status lookup with VietQR sandbox callback API. Adding or removing provider-specific status actions requires changing this shared service.	Split into PaymentTransactionStatusApi and VietQrSandboxCallbackApi.
PaymentModule	Binh NT	Medium risk. VietQR controller, guard, config, service, and HTTP client are registered directly inside PaymentModule. Changing	Create VietqrModule that owns VietQR providers and exports only a narrow service/port. PaymentModule should import VietqrModule.

		VietQR wiring or disabling sandbox endpoints requires editing the payment module.	
ProductService	HieuBT	The constructor hardcodes the list of handlers = [new BookHandler(), new CdHandler(),...]. When adding a new product type like Clothing, this constructor must be modified directly.	Inject the IProductHandler[] array via the constructor instead of manually instantiating it with the new keyword.
CreateProductDto	HieuBT	Adding a new product type requires modifying this DTO to include its specific properties.	Use polymorphism to let new product types define custom DTOs extending BaseCreateProductDto.
ProductDetailComponent.summarySpecs() (client)	NghiaTT	Violation. Behavior is selected by a switch-case over product.type (BOOK / NEWSPAPER / CD / DVD). Adding a new product type (e.g. Clothing, E-Book) forces modifying this method, so the class is not closed for modification.	Delegate per-type summary to a type→spec configuration map or to ProductSpecificInfoComponent, so a new product type plugs in without editing summarySpecs().
ProductDetailMapper.mapProductDetail() (api – productdetail.service.ts)	NghiaTT	Violation. The correct mapper is resolved with an if/else chain on type (BOOK / NEWSPAPER / CD / DVD); every new product type requires editing this method.	Replace the if/else chain with a type→mapper registry map so new types register their own mapper without modifying existing code.
ProductDetailMapper.mapSpecificInfo() (api – application/mappers)	NghiaTT	Partial violation. It uses a Record map – better than if/else – but the map literal still must be edited for every new product type, so the class is not fully closed for modification	Register specific-info mappers through a handler registry / DI so new product types are added by registration, leaving the mapper closed for modification.
Architecture / PaypalGatewayAda	Tu NHT	Thanks to the PaymentGateway interface, the system is open for extension (e.g., adding	- Move API endpoint paths (/v1/..., /v2/...) to configuration files or

pter		Stripe, VNPay) by adding new Adapters without modifying existing code. However, PaypalService hardcodes API endpoints (/v1/oauth2/token, /v2/checkout/orders). If PayPal upgrades to v3, this class must be modified directly.	environment variables. - Build more generalized API call methods in the Service so that adding new PayPal endpoints doesn't require rewriting the entire try-catch block and headers.
------	--	--	--

3. Liskov Substitution Principle (LSP)

Modules	PIC	Description	Improvement Direction
PlaceOrderPaymentAdapter vs FakePlaceOrderPaymentAdapter	Khanh NT	Both implement PlaceOrderPaymentPort.markPaidAndPendingProcessing(). The real adapter decrements stock, updates order status in the DB, and sends a confirmation email. The fake adapter only logs a TODO. They are not substitutable: using the fake in production leaves orders unprocessed and customers without confirmation. The interface does not specify postconditions, making the contract ambiguous.	Document the postconditions of the interface (order marked paid, stock decremented). Extract email notification outside the interface boundary so both implementations can satisfy the core contract without duplicating notification logic.
ConfirmOrderDto extends SubmitDeliveryInfoDto	Khanh NT	ConfirmOrderDto inherits items + deliveryInfo from SubmitDeliveryInfoDto purely for field reuse, not because it is a true "SubmitDeliveryInfo". The subclass adds mandatory fields (transactionId, paymentMethod) that tighten	Replace inheritance with composition: ConfirmOrderDto directly declares items, deliveryInfo, transactionId, and paymentMethod. Removes the false "is-a" relationship between two different steps of the order flow.

		preconditions beyond what the parent declares. Code expecting a SubmitDeliveryInfoDto cannot safely receive a ConfirmOrderDto - substitution breaks the parent's contract.	
VietqrClient vs VietqrHttpClient	Binh NT	Mostly compliant. VietqrHttpClient satisfies the VietqrClient contract. Remaining risk: generateQRCode() response allows both root-level fields and nested data, so future implementations may return inconsistent shapes while still matching the loose interface.	Normalize VietQR HTTP response inside the adapter. Make VietqrClient.generateQRCode() return one stable response shape.
PaymentFlowStrategy vs VietQrPaymentFlowService	Binh NT	Mostly compliant with some contract ambiguity. VietQrPaymentFlowService implements the strategy, but it also persists snapshots and triggers sandbox callback. Other payment strategies may not support these side effects, so callers could incorrectly expect every strategy to behave like VietQR.	Document strategy postconditions clearly. Keep snapshot persistence and sandbox callback outside the common PaymentFlowStrategy contract.
PaypalGatewayAdapter & PaypalService	Tu NHT	The Adapter implements PaymentGateway and can be substituted for other gateways. However, PaypalService throws framework-specific HTTP technical errors (UnauthorizedException, InternalServerErrorException from NestJS). If upper layers (Core Domain) call the PaymentGateway and	Catch HTTP errors at the Service or Adapter layer and map them to standard Domain Exceptions (e.g., PaymentProviderException, PaymentValidationException). Prevent framework HTTP Exceptions from leaking into the Domain/Application layers.

		receive HTTP framework errors instead of Domain Exceptions, polymorphism and abstraction are broken.	
--	--	--	--

4. Interface Segregation Principle (ISP)

Modules	PIC	Description	Improvement Direction
PlaceOrderPaymentAdapter depends on MailService	Khanh NT	The adapter only calls mailService.sendMail() but is forced to depend on the entire MailService, which also exposes sendInvitation(), sendOtpCode(), and emailTransport() — methods entirely unrelated to order placement.	Define a narrow IOrderMailSender { sendMail(dto): Promise<void> }. Inject IOrderMailSender. MailService implements it. The adapter is decoupled from invitation and OTP email logic.
DeliveryInfoValidator - over-exposed API	Khanh NT	The validator publicly exposes 5 individual static methods (validateReceiverName, validatePhoneNumber, validateProvince, validateAddress, validateShippingInstructions) in addition to validate(). PlaceOrderBeService only ever calls the aggregate validate(). All callers are forced to know about and depend on a wider surface than they need.	Change the individual validation methods to private. Only validate() remains public. Granular methods are tested through the aggregate or via test-only helpers.
CartItemDto - unused fields	Khanh NT	CartItemDto carries price?: number and weight?: number that are never read by normalizeItems(). Prices and weights are always fetched from the DB inside buildInvoicePreviewFromPro	Remove price and weight from CartItemDto. Keep only productId and quantity. Define a separate DTO for use cases that genuinely need client-side price or weight.

		ducts(). Every consumer of the DTO is exposed to fields it does not use.	
PaymentStatusApiService	Binh NT	Low risk. Callers that only need getLatestByOrderId() are also exposed to requestVietQrSandboxCallback(). This creates a wider interface than most production callers need.	Split into PaymentTransactionStatusApi for production status lookup and VietQrSandboxCallbackApi for test callback behavior.
VietqrClient	Binh NT	Low risk. The interface currently includes authentication, QR generation, and sandbox callback. Production QR generation users do not need the sandbox callback method.	If the interface grows further, split into VietqrAuthClient, VietqrQrClient, and VietqrSandboxClient.
CheckoutDraftService		Low risk. Consumers that only need draft persistence are exposed to validation and DTO mapping methods too.	Split into CheckoutDraftStorage, CheckoutDraftValidator, and CheckoutDraftMapper if more callers start depending on only part of the service.
IProductHandler	HieuBT	The create() method relies on a bloated God dto, forcing BookHandler to unnecessarily depend on irrelevant fields from other product types.	Update create() signature to use type-specific dtos or a generic IProductHandler<T>.
ProductQueryRepository (api – domain interface)	NghiaTT	Low risk. The interface bundles findProducts() (list) and findProductById() (detail). A detail-only consumer such as GetProductDetailUseCase still depends on the list method it never uses, exposing a wider surface than needed.	If more read methods are added, split into ProductListQueryRepository and ProductDetailQueryRepository so each consumer depends only on the query it actually uses.
PaymentGateway (Interface)	Tu NHT	Based on the Adapter, the PaymentGateway interface aggregates createPayment,	Segregate PaymentGateway into smaller, more specific interfaces if necessary, such as

		confirmPayment, and refundPayment. If a future payment method doesn't support refunds (e.g., manual Bank Transfer), its implementing class would have to leave refundPayment empty or throw an unsupported error, violating ISP.	IPaymentCreator, IPaymentConfirmer, and IPaymentRefunder. Adapters should only implement the interfaces they actually support.
--	--	--	--

5. Dependency Inversion Principle (DIP)

Modules	PIC	Description	Improvement Direction
PlaceOrderService → ShippingFeeService	Khanh NT	High-level business logic depends directly on the concrete ShippingFeeService class - no interface. Swapping the shipping strategy requires changing the constructor and injection site in the service itself.	Introduce IShippingFeeCalculator. Inject the abstraction token SHIPPING_FEE_CALCULATOR in PlaceOrderService. ShippingFeeService implements the interface.
PlaceOrderBeService → MailService	Khanh NT	Direct dependency on concrete MailService - no interface. Switching to a different notification channel (SMS, Zalo) requires changing the constructor and dependency.	Introduce INotificationService. Inject it instead of MailService.
PlaceOrderService → DeliveryInfoValidator (static)	Khanh NT	The service calls DeliveryInfoValidator.validate (...) as a static concrete class - non-injectable and non-mockable. High-level business logic is hard-coupled to a specific validation implementation; unit tests cannot substitute a mock validator.	Add @Injectable() to DeliveryInfoValidator, define IDeliveryInfoValidator, and inject it into PlaceOrderBeService via the constructor.
PlaceOrderPaymentA	Khanh NT	The adapter injects concrete	Inject IOrderMailSender (see

dapter → MailService		MailService - no interface. Changing the email provider forces modification of the adapter.	ISP above). Decouples the adapter from the concrete mail implementation.
PlaceOrderModule - concrete provider registration	Khanh NT	ShippingFeeService is registered directly as a provider without an abstraction token. Any consumer swapping the strategy must modify both the module providers array and PlaceOrderBeService's constructor.	Register as { provide: SHIPPING_FEE_CALCULATOR, useClass: ShippingFeeService }. Changing the implementation only requires updating useClass in this one place.
VietqrService -> process.env	Binh NT	Partial violation. VietqrService directly reads environment variables for credentials and QR settings. This hides configuration dependencies and makes tests/config substitution harder.	Introduce typed VietqrConfigService and inject it into VietqrService.
VietqrHttpClient -> axios/process.env	Binh NT	Medium risk. The HTTP client depends directly on concrete axios and reads base URL from process.env.	Inject Nest HttpService or an HTTP client abstraction, and inject typed config for base URL.
PaymentService -> VietqrService/Paypal Service	Binh NT	Clear violation. High-level payment workflow depends directly on concrete provider services. Swapping or adding providers requires constructor and logic changes.	Inject PaymentGatewayPort strategies through provider tokens or a gateway registry.
VietqrCallbackAuthGuard -> process.env	Binh NT	Medium risk. The guard reads VIETQR_CALLBACK_TOKEN directly from environment.	Inject config service into the guard.
VietQrPaymentFlowService -> localStorage	Binh NT	Medium risk. The service directly uses browser localStorage, making snapshot persistence hard to substitute in tests or SSR.	Inject PaymentSnapshotStore abstraction.

PaymentPageFacade -> qrcode	Binh NT	Low risk. The facade directly imports the concrete qrcode library for QR image generation.	Introduce QrImageService and inject it.
DeliveryComponent -> VietQrPaymentFlowService/PlaceOrderApiService	Khanh NT	Medium risk. The UI component depends directly on concrete workflow/API services instead of page-level use-case abstractions.	Inject DeliveryFormFacade / PlaceOrderCheckoutFacade; component should bind UI state and delegate workflows.
CheckoutDraftService -> localStorage	Khanh NT	Low risk. The service directly depends on browser storage.	Inject CheckoutDraftStorage abstraction
ProductService.constructor	HieuBT	ProductService is tightly coupled to concrete low-level classes by instantiating them with the new keyword.	Use a custom NestJS Dependency Injection token to provide the array of handlers at the Module level and inject them into the Service.
ProductDetailService → static helpers (apis – productdetail.services)	NghiaTT	Violation. The orchestrating service calls ProductDetailValidator, ProductDetailMapper and ProductAvailabilityService as static concrete classes – noninjectable and non-mockable. High-level retrieval logic is hard-coupled to concrete helper implementations, so unit tests cannot substitute mocks.	Define interfaces (IProductDetailValidator, IProductDetailMapper, IProductAvailabilityService) and inject them through the constructor instead of calling static methods.
ProductDetailPageComponent → ProductApiService / CartStoreService (client)	NghiaTT	Low risk. The page/component depend directly on concrete service classes rather than a use-case abstraction. Swapping the data source or cart store requires touching the component.	Depend on a narrow ProductDetailFacade / interface token; keep the component bound to UI state and delegate data + cart operations.
PaypalService & PaypalController	Tu NHT	PaypalService depends directly on a low-level concrete library (axios) by importing it directly rather	- Replace axios by injecting HttpService via the constructor (using the NestJS

		than injecting an HTTP Client abstraction. Furthermore, PaypalGatewayAdapter and PaypalController depend on the concrete implementation of PaypalService instead of an abstraction.	IoC Container). - (Optional/Advanced) Create an IPaypalService interface and have PaypalGatewayAdapter depend on this interface instead of the concrete class, which makes unit testing (mocking) much easier.
--	--	---	--